

Resumo

Introdução à Utilização de
Git & Github

Hygor Melo de Sena

11 de janeiro de 2026

Sumário

1	Capítulo 1 ► O que é Git? O que é versionamento?	3
2	Capítulo 2 ► O que é GitHub? Para que serve?	4
3	Capítulo 3 ► A evolução do Git e GitHub	5
4	Capítulo 4 ► Instalações e configurações importantes	6
5	Capítulo 5 Repetições	7
6	Capítulo 6 Listas, dicionários, tuplas e conjuntos	8
6.1	Trabalhando com índices	8
6.2	Cópia e fatiamento de listas	8
6.3	Tamanho de listas	8
6.4	Adição de elementos	8
6.5	Remoção de elementos da lista	8
6.6	Usando listas com filas	8
6.7	Uso de listas com pilhas	8
6.8	Pesquisa	8
6.9	Usando <i>for</i>	9
6.10	Range	9
6.11	Enumerate	9

1 Capítulo 1 ► O que é Git? O que é versionamento?

Git é um dos sistemas de controle de versões de programas, ou seja, ele é um sistema de controle de versão (VCS). Dessa afirmação tiramos o significado de versionamento. Versionamento trata-se simplesmente de versões que um *software* pode sofrer durante seu processo de desenvolvimento como se pode ver abaixo:

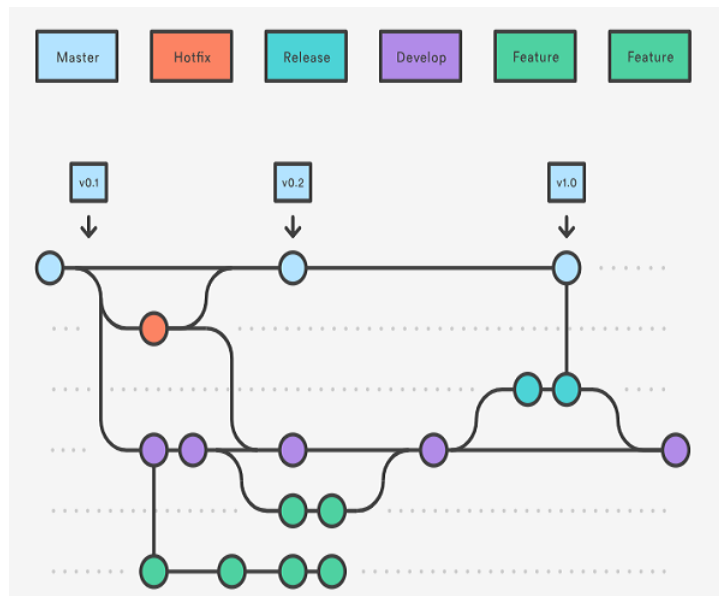


Figura 1: Fluxograma de Versionamento Git

Na figura acima também podemos ver o ciclo de versionamento do Git. Temos a versão Master, Hotfix, Develop, Release, Feature. Master tem por finalidade um branch de produção, Develop é branch de integração contínua do desenvolvimento, Feature é desenvolvimento de novas funcionalidades específicas, Release é preparar uma versão candidata à produção, Hotfix é correção urgente em produção. Confira a tabela abaixo:

Branch	Objetivo Principal	Estável	Origem	Destino Final
Master	Produção	Sim	Release/Hotfix	—
Develop	Integração do desenvolvimento	Parcial	Master	Release
Feature	Nova funcionalidade	Não	Develop	Develop
Release	Estabilização de versão	Quase	Develop	Master + Develop
Hotfix	Correção urgente	Sim	Master	Master + Develop

2 Capítulo 2 ► O que é GitHub? Para que serve?

GitHub é uma plataforma online de hospedagem e colaboração de código baseada no Git, utilizada para versionar, compartilhar, revisar e gerenciar projetos de software de forma organizada e rastreável. Em termos práticos, o GitHub permite:

- Armazenar código-fonte em repositórios remotos.
- Controlar versões (histórico completo de alterações).
- Trabalhar em equipe, com múltiplas pessoas atuando no mesmo projeto sem sobrecrever código.
- Criar e revisar mudanças por meio de *branches*, *commits* e *pull requests*.
- Documentar projetos (README, wikis).
- Automatizar processos com CI/CD (Github Actions)
- Gerenciar tarefas com *issues*, *milestones* e projetos.

Em uma frase: GitHub é o ambiente onde projetos versionados com Git vivem, evoluem e são colaborados publicamente ou de forma privada.

3 Capítulo 3 ► A evolução do Git e GitHub

Git (2005) nasceu da necessidade e da raiva:

Linus Torvalds ficou sem ferramenta boa depois da briga com a BitKeeper, criou algo rápido, poderoso, distribuído e com integridade de dados fortíssima.

GitHub (2008) nasceu da pergunta:

"E se a gente pegasse esse Git incrível... e colocasse uma interface web bonita + rede social + colaboração fácil em cima dele?"

Resultado: GitHub transformou o Git de "ferramenta de linha de comando para nerds do kernel" em algo que todo mundo usa (empresas, iniciantes, open-source, times enormes, educação...). Curiosidades legais O primeiro commit do Git (7 de abril de 2005) foi um README bem simples O apelido inicial do Git dado pelo próprio Linus: "the information manager from hell".

GitHub demorou alguns anos para ficar famoso, mas quando explodiu... foi muito rápido Hoje em dia muita gente fala "Git" quando na verdade quer dizer "GitHub" (e vice-versa) Resumindo em uma frase: Git resolveu o problema técnico de controle de versão distribuído. GitHub resolveu o problema social de como milhões de programadores trabalham juntos.

4 Capítulo 4 ► Instalações e configurações importantes

5 Capítulo 5 Repetições

6 Capítulo 6 Listas, dicionários, tuplas e conjuntos

6.1 Trabalhando com índices

De forma leiga, índices são números que fazem referência a um elemento de uma lista, veja o programa abaixo:

```
1      números = [0, 0, 0, 0, 0]
2      x = 0
3      while x < 5:
4          números[x] = int(input(f"Número {x + 1}: "))
5          x += 1
6
7      while True:
8          escolhido = int(input(f"Que posição você quer imprimir (0 para
          ↪ sair): "))
9
10         if escolhido == 0:
11             break
12
13         print(f"Você escolheu o número {números[escolhido - 1]}")
```

6.2 Cópia e fatiamento de listas

6.3 Tamanho de listas

6.4 Adição de elementos

6.5 Remoção de elementos da lista

6.6 Usando listas com filas

6.7 Uso de listas com pilhas

6.8 Pesquisa

Podemos pesquisar se um elemento está ou não em uma lista, verificando do primeiro ao último elemento se o valor procurado estiver presente:

```
#Pesquisa Sequencial

L = [15, 7, 27, 39]
p = int(input("Digite o valor a procurar:"))
achou = FALSE
x = 0

while x < len(L):
    if L[x] == p:
        achou = TRUE
```



```

break
x += 1

if achou:
    print(f"{p} achado na posição {x}")

else:
    print(f"{p} não encontrado")

```

6.9 Usando for

A instrução `for` funciona de forma parecida a *while*, mas a cada repetição utiliza um elemento diferente da lista.

A cada repetição, o próximo elemento da lista é utilizado, o que se repete até o fim da lista. Vamos escrever um programa que utilize **for** para imprimir todos os elementos de uma lista:

```

L = [8, 9, 15]
for e in L:
    print(e)

```

6.10 Range

A função **range** é um gerador de números que possui início (i), fim **aberto** (f) e um compasso (c), onde este é a uma contagem gradativa $\{c = c + (n + 1), \forall n \in \mathbb{N}\}$. Veja no exemplo abaixo:

```

for t in range(i, f, c):
    print(t, end=" ")
print()

```

Observe que poderemos substituir i por 1, f por 100 e c por 2, por exemplo. Isso vai nos gerar número ímpares. ¹

6.11 Enumerate

Com a função **enumerate** podemos ampliar as funcionalidades de **for** facilmente. Vejamos como imprimir uma lista, em que teremos o índice entre colchetes e o valor à sua direita:

```

L = [5, 9, 13]
x = 0
for e in L:
    print(f"[{x}] {e}")
    x+=1

```

¹Lembre-se que f é aberto, ou seja, o último número gerado na verdade será 99 e não 100.

Veja o mesmo programa, mas utilizando a função **enumerate**:

```
L = [5, 9, 13]
for x, e in enumerate(L):
    print(f"[{x}] {e}")
```